

Algorithm: Universal Keyword Identification Pipeline

Author: Ece Nur Kızıklı

Date: 2026-06-18

Version: 1.0

Phase 1 — Data Preparation & Universal Candidate Detection ([analyzer.py](#))

1. Load Raw Data
Read the Apple Search Ads report CSV file ([data.csv](#)) containing search terms, countries, and popularity scores.
2. Filter by Popularity
Remove all rows where Search Popularity < 20 to eliminate low-volume, insignificant search terms.
3. Map Countries to ISO Alpha-2 Codes
For each unique country name in the dataset, resolve it to its ISO 3166-1 alpha-2 code (e.g., "United States" → "US") using pycountry fuzzy matching, with manual overrides for ambiguous names (e.g., "Korea" → "KR").
4. Build Country-to-Language Lookup
For each unique country, retrieve its spoken languages using the countryinfo library. This produces a mapping like "Switzerland" → ["de", "fr", "it", "rm"].
5. Identify Universal Candidates (Country Threshold)
Group all search terms by name and count the number of distinct countries each term appears in. Filter to keep only terms that appear in 5 or more countries. These are the universal keyword candidates.
6. Detect Keyword Language via Keyword Metadata API
For each candidate keyword, call the keyword-metadata API ([/keyword-metadata](#)) using up to 3 country codes to retrieve the [languageCode](#) of the keyword itself. This tells us what language the keyword originates from (e.g., "pizza" → "it", "sushi" → "ja"). API calls are parallelized with 8 threads and rate-limited at 50ms between retries per term.
7. Compute Language Coverage Metrics
For each candidate keyword, calculate two metrics:
 - Top Shared Language Percentage ([top_shared_lang_pct](#)): Find the single language that is spoken in the most countries where this keyword appears. Divide that count by the total number of countries

the keyword appears in. Example: If "running" appears in 10 countries and English is spoken in 7 of them → $7/10 = 0.70$.

- Search Term Origin Language Coverage
([search_term_origin_lang_coverage_pct](#)): Count how many of the keyword's countries speak the keyword's own detected language. Divide by total countries. Example: If "laufen" (detected as German) appears in 8 countries and German is spoken in 3 of them → $3/8 = 0.375$.

8. Apply Universal Flag Criteria

A keyword is flagged as Universal = True if ALL of the following conditions are met:

1. It appears in ≥ 5 countries
2. [top_shared_lang_pct](#) < 0.50 (no single language dominates more than half the countries)
3. [search_term_origin_lang_coverage_pct](#) < 0.50 (the keyword's own language is not spoken in more than half the countries)

The logic: if a keyword appears across many countries and no common language explains its spread, it is likely a truly universal keyword.

9. Export Results

Write all universal keywords to [universal_keywords_output.csv](#).

Phase 2 (Optional) — Brand Filtering via Keyword Metadata API ([brand_check.py](#))

10. Load Universal Keywords

Read the universal keywords from [universal_keywords_output.csv](#).

11. Build Country Code Priority List

For each keyword, collect its associated country codes and prepend a priority list ["US", "GB", "TR", "KR", "JP"]. Select up to 5 codes per keyword.

12. Check Brand Status via keyword-metadata API

For each keyword, call the keyword-metadata API across up to 5 country codes. If any response contains a non-null [brandApp](#) field, the keyword is marked as a brand keyword. Parallelized with 8 threads.

13. Split and Export

- Keywords with [Is_Brand](#) = True → [universal_keywords_with_brand.csv](#)
- Keywords with [Is_Brand](#) = False → [universal_keywords_final.csv](#)

Phase 3 (Optional) — Brand Filtering via Internal Database ([brand_recheck.py](#))

14. Load Remaining Non-Brand Keywords

Read [universal_keywords_final.csv](#).

15. Batch Query Internal Database

Connect to the internal MySQL database and query the keyword table in batches of 500. Any keyword with a non-null [brand_app](#) column is flagged as brand.

16. Split and Export

- Brand keywords → [brands_caught_recheck.csv](#)
- Final clean universal keywords → [universal_keywords_final.csv](#) (overwritten)

Error Handling Summary

Phase	On API/DB Failure	Behavior
Phase 1	Skip, label "unknown_term_lang"	Fail-open: keyword proceeds with unknown language
Phase 2	Skip, treat as non-brand	Fail-open: keyword passes through
Phase 3	Unhandled exception	Fail-closed: script crashes

Note: Keywords with ["unknown_term_lang"](#) will have [search_term_origin_lang_coverage_pct](#) = 0 since no country speaks ["unknown_term_lang"](#). This biases the keyword toward being flagged as universal. In practice, this is acceptable as a fail-open behavior, false positives are caught by the subsequent brand filtering phases.

Pipeline Summary

[data.csv](#)

→ [\[analyzer.py\]](#) => [universal_keywords_output.csv](#)

→ [\[brand_check.py\]](#) => [universal_keywords_final.csv](#) +
[universal_keywords_with_brand.csv](#)

→ [\[brand_recheck.py\]](#) => [universal_keywords_final.csv](#) (cleaned) +
[brands_caught_recheck.csv](#)